

Automated MLOps Safety Monitoring Platform

Final Year Project Report

Umar Nadeem Goraya
22i-1136
FAST NUCES

June 14, 2026

Contents

1	Introduction	2
1.1	Project Overview	2
1.2	Objectives	2
2	System Architecture	3
2.1	Technology Stack	3
2.2	Pipeline Flow	3
3	System Validation & Testing	4
3.1	Model Serving & API Layer	4
3.2	Infrastructure Health	4
3.3	Real-Time Observability & Stress Testing	5
3.4	Automated Incident Routing	6
3.5	CI/CD Automation	7
4	Conclusion	8

Chapter 1

Introduction

1.1 Project Overview

The deployment of Machine Learning models into production environments introduces unique challenges beyond traditional software engineering. This project introduces an Automated MLOps Safety Monitoring Platform designed to track model performance, infrastructure health, and data integrity in real-time.

1.2 Objectives

- Deploy a containerized Machine Learning inference API.
- Implement real-time telemetry extraction using time-series databases.
- Visualize system stress, hardware bottlenecks, and data drift.
- Automate incident management and notification routing.
- Establish a Continuous Integration and Continuous Deployment (CI/CD) pipeline.

Chapter 2

System Architecture

2.1 Technology Stack

The system architecture leverages a modern, cloud-native technology stack deployed on AWS EC2 instances.

- **Serving Layer:** FastAPI, Uvicorn, Python
- **Machine Learning:** Scikit-Learn (Random Forest Classifier)
- **Observability:** Prometheus (Metrics Scraping), Grafana (Visualization)
- **Alerting:** Alertmanager, Slack Webhooks
- **Infrastructure:** Docker, Docker Compose, GitHub Actions

2.2 Pipeline Flow

Incoming requests are processed by the FastAPI container, which executes the Random Forest inference. Prometheus scrapes the ‘/metrics’ endpoints at defined intervals, storing time-series data. Grafana queries Prometheus to visualize the system state, while Alertmanager listens for threshold breaches to trigger Slack notifications.

Chapter 3

System Validation & Testing

This chapter details the rigorous testing methodologies applied to the system to ensure production readiness.

3.1 Model Serving & API Layer

The inference engine requires exact payload formatting to function. The system was validated to ensure it correctly parses inputs and returns valid predictions.

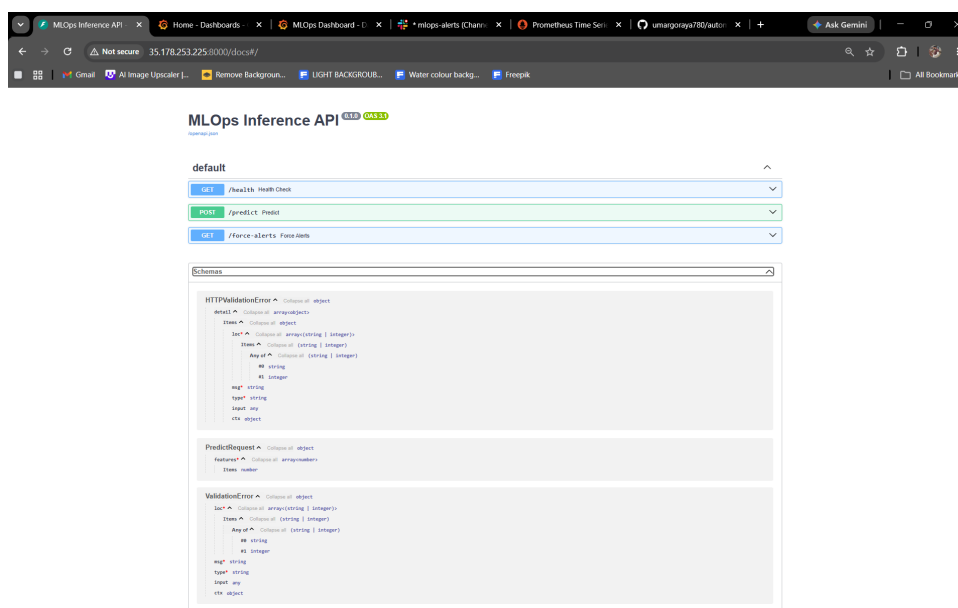


Figure 3.1: Swagger UI Documentation showcasing the /health, /predict, and /force-alerts endpoints.

3.2 Infrastructure Health

Prometheus was configured to actively discover and scrape the FastAPI containers.

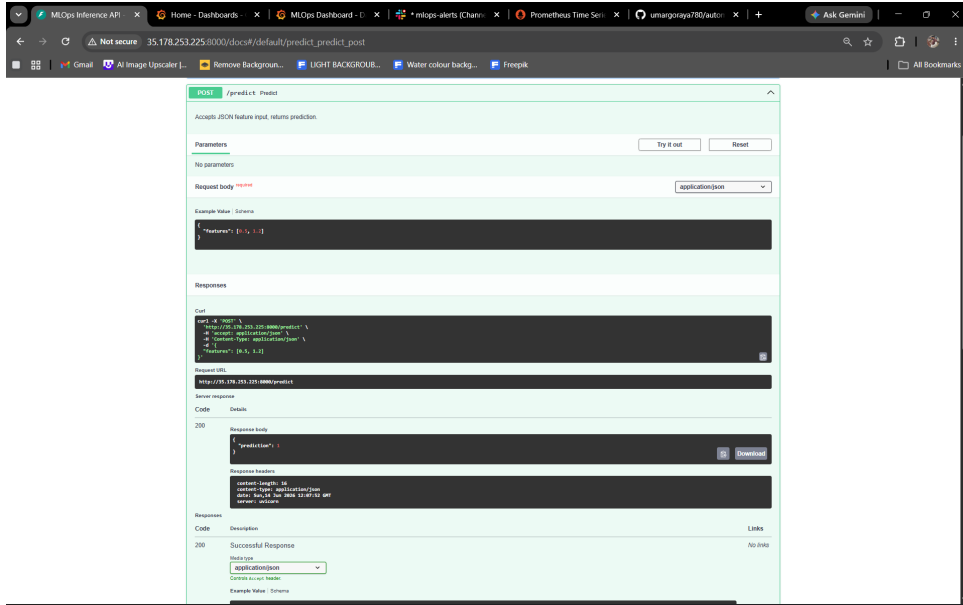


Figure 3.2: Successful Inference Execution via POST `/predict` showing a Status 200 OK.

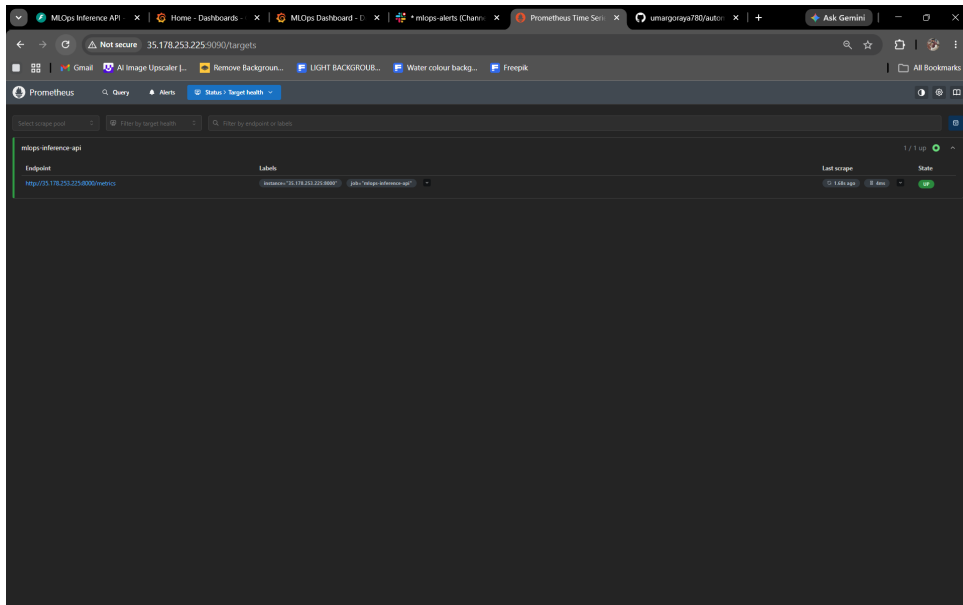


Figure 3.3: Prometheus Targets console confirming successful connection to the inference container.

3.3 Real-Time Observability & Stress Testing

To validate the observability layer, a custom Python-based multi-threaded stress test was engineered. The script generated a sustained 45-second wave of traffic targeting the model to evaluate resource exhaustion.

Furthermore, the system successfully identified simulated infrastructure failures (Data-Lake connection drops) and data drift anomalies.

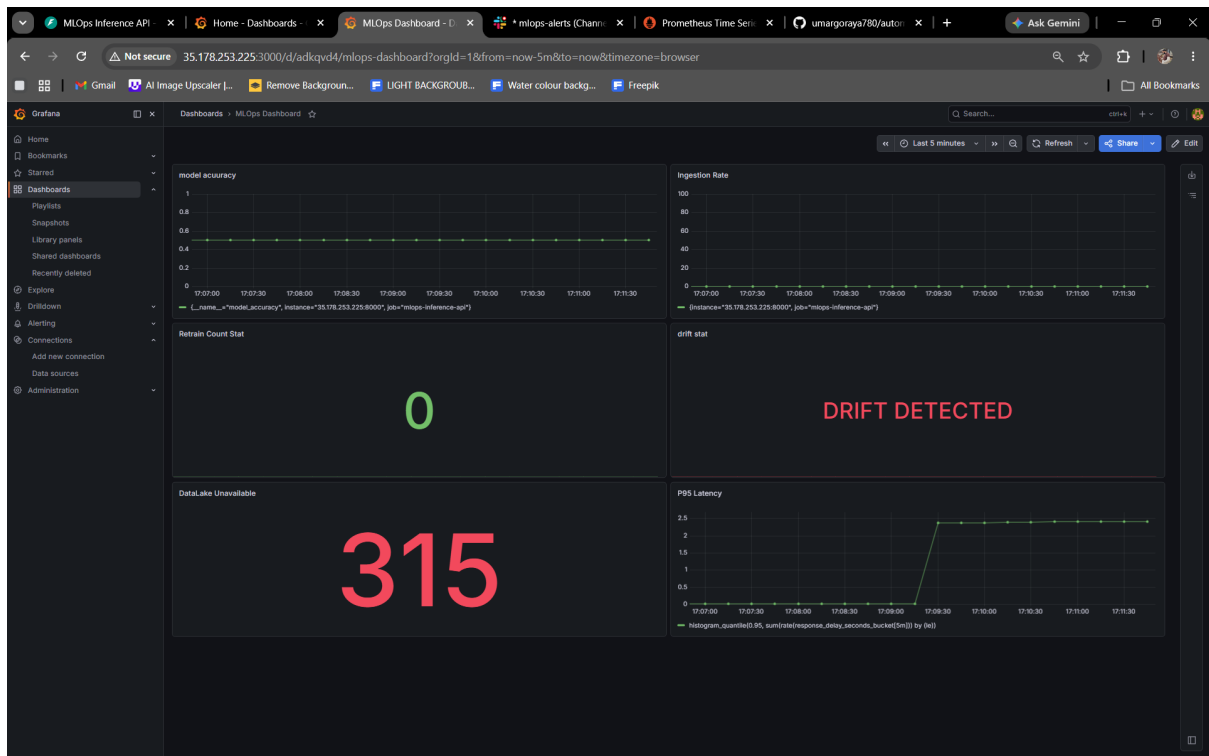


Figure 3.4: Grafana Dashboard visualizing the P95 Latency spike during the sustained load test.

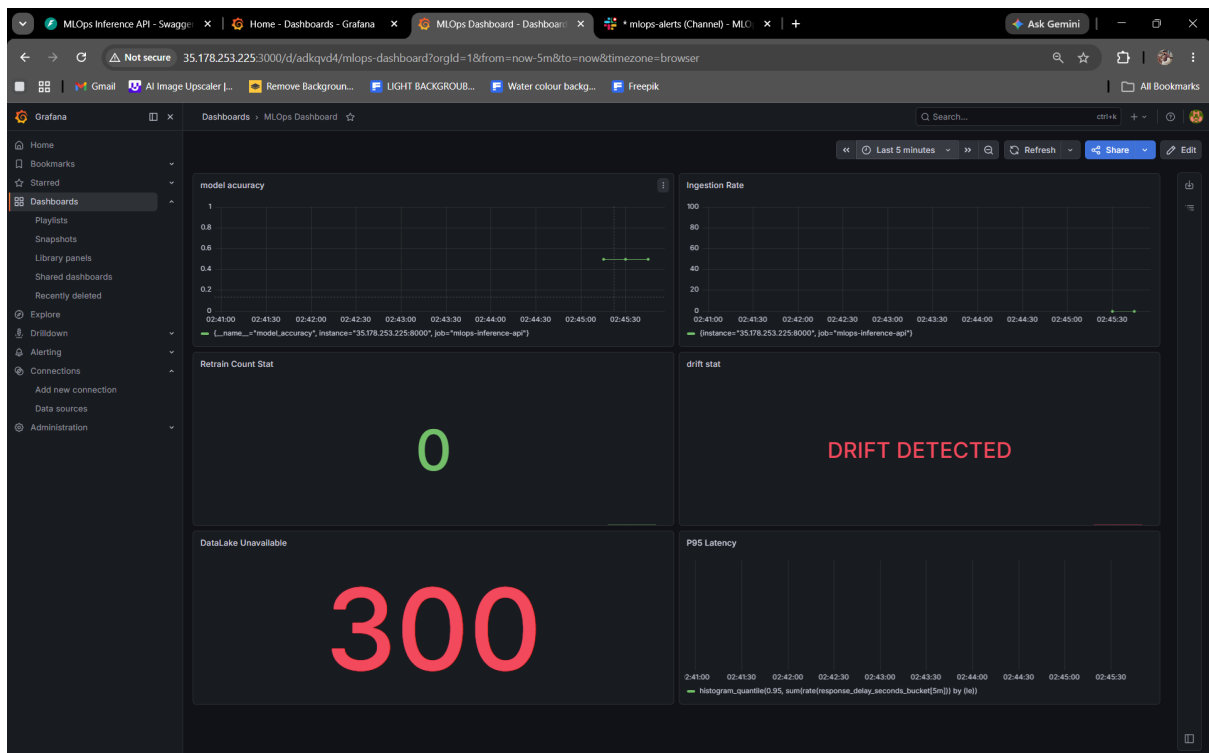


Figure 3.5: Dashboard capturing concurrent DataLake failures and Drift Detection warnings.

3.4 Automated Incident Routing

System degradation is actively routed to the engineering team. Upon triggering the ‘/force-alerts‘ tripwire, the Alertmanager successfully routed the payload to the design-

nated communication channels.

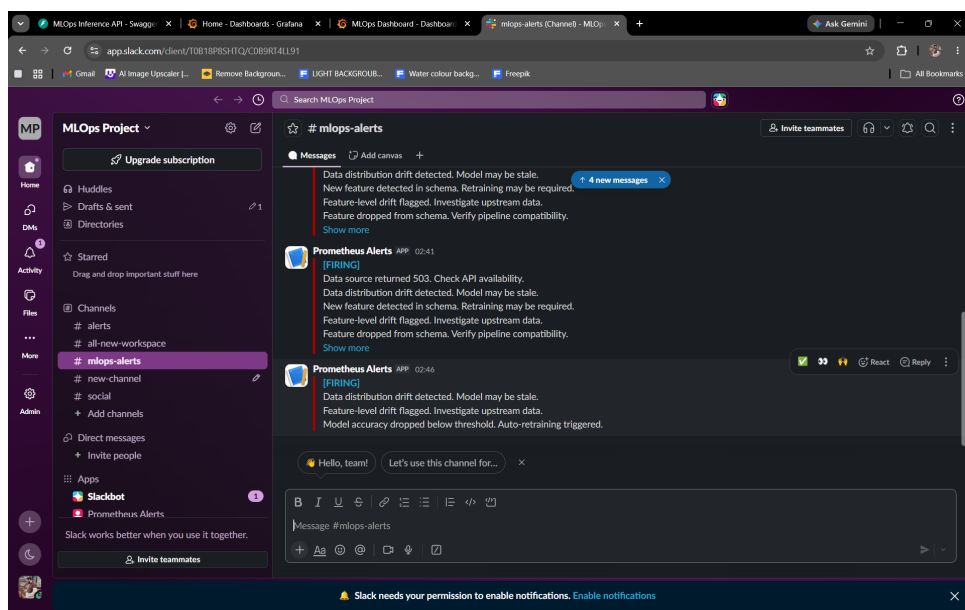


Figure 3.6: Automated incident notification delivered via Slack Webhook.

3.5 CI/CD Automation

The deployment lifecycle is fully automated. Code pushed to the repository triggers a GitHub Actions workflow that executes the deployment script on the AWS EC2 instance.

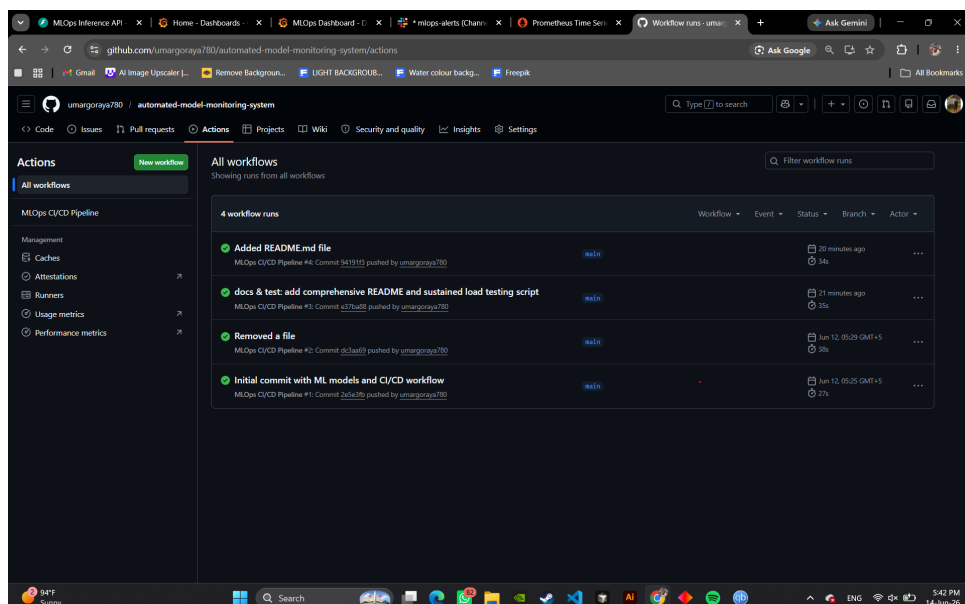


Figure 3.7: GitHub Actions workflow demonstrating a successful automated deployment.

Chapter 4

Conclusion

The Automated MLOps Safety Monitoring Platform successfully demonstrates the integration of machine learning models with robust, cloud-native DevOps practices. By implementing automated CI/CD, real-time observability, and closed-loop alerting, the architecture ensures models remain performant, accurate, and stable in production environments.